

Manual de Descomposición Modular

Parte II — Teoría de Conjuntos

Lógica 101

Sebastián Castillo Martínez
Gustavo Adrián Joya Rodríguez
Jesus Daniel Marín Terrazas

TABLA DE CONTENIDO

Introducción	3
Arquitectura General del Sistema (Parte II)	4
Diagrama de Módulos	5
Módulo 3: Backend — Teoría de Conjuntos	6
Módulo 4: Frontend — Tab Conjuntos	8
Algoritmo de Evaluación de Expresiones	9
Algoritmo de Operación Guiada	10
Manejo de Errores (Parte II)	11
Estructuras de Datos Utilizadas	12
Conclusión Técnica	13

INTRODUCCIÓN

Este documento extiende el Manual de Descomposición Modular de la Suite de Validación Lógica hacia la **Parte II del proyecto integrador**: la Calculadora de Teoría de Conjuntos. Se describe la arquitectura del nuevo módulo, las funciones del backend matemático, la interfaz gráfica ampliada y los algoritmos que gestionan las operaciones sobre conjuntos.

La Parte II conserva la separación de responsabilidades establecida en la Parte I: el motor lógico-matemático permanece desacoplado de la vista. Adicionalmente, el sistema fue reempaquetado como **LyED Toolkit**, una aplicación con pestañas (tabs) que alberga ambos módulos dentro de una única ventana unificada.

Entre los aspectos técnicos destacados de esta entrega se encuentran: la gestión dinámica de conjuntos (número ilimitado), la carga de datos desde archivos .txt, el soporte de elementos anidados a dos niveles, un modo de operación guiado mediante dropdowns y un motor de expresiones libres con evaluación segura.

ARQUITECTURA GENERAL DEL SISTEMA (PARTE II)

La **Parte II** se integra como una segunda pestaña dentro de la misma clase **SuiteLogica**. La arquitectura mantiene el patrón modular heredado: un módulo de backend puro (funciones libres) y un módulo de frontend (métodos de la clase). Ambos módulos coexisten en el mismo archivo fuente, diferenciados por bloques de comentarios y responsabilidades claramente definidas.

Capa	Componente	Responsabilidad principal
Backend	Módulo 3 — Conjuntos	Parseo, operaciones matemáticas, evaluación de expresiones
Frontend	Módulo 4 — Tab Conjuntos	Tarjetas de conjuntos, modos de operación, renderizado de resultados
Integración	CTkTabview	Unifica Parte I y Parte II en una sola ventana tabbed

El flujo de datos en la Parte II sigue la misma filosofía que en la Parte I: el usuario interactúa exclusivamente con la interfaz gráfica, que delega toda lógica matemática al backend. Los conjuntos se almacenan internamente como objetos **set** de Python y solo son formateados a texto en el momento del renderizado.

DIAGRAMA DE MÓDULOS

MÓDULO 3: Backend — Conjuntos	MÓDULO 4: Frontend — Tab Conjuntos
• <code>parse_elemento(s)</code> • <code>cargar_conjunto_desde_texto(texto)</code> • <code>formatear_elemento(e)</code> • <code>formatear_conjunto(s)</code> • <code>evaluar_expresion_conjuntos(expr, conjs)</code> • <code>operacion_guiada(a, b, operacion)</code>	• <code>_construir_tab_conjuntos()</code> • <code>_agregar_tarjeta(nombre)</code> • <code>_agregar_conjunto()</code> • <code>_eliminar_ultimo_conjunto()</code> • <code>_construir_modos_guiado()</code> • <code>_construir_modos_expresion()</code> • <code>_calcular_conjuntos()</code> • <code>_mostrar_resultado_conjuntos()</code> • <code>_mostrar_error_conj()</code> • <code>_cargar_archivo(nombre)</code> • <code>_aplicar_texto(nombre)</code>

Flujo de comunicación entre módulos:

1. El usuario define conjuntos (texto o archivo .txt) en las tarjetas del panel izquierdo.
2. Al hacer clic en **Aplicar texto** o **Cargar archivo**, el frontend invoca `cargar_conjunto_desde_texto()` del backend.
3. El usuario elige una operación en el modo Guiado (dropdowns) o escribe una expresión libre.
4. Al presionar **Calcular**, el frontend llama a `operacion_guiada()` o `evaluar_expresion_conjuntos()`.
5. El resultado (objeto set) es devuelto al frontend, que lo renderiza junto con la tabla de pertenencia.

MÓDULO 3: BACKEND — TEORÍA DE CONJUNTOS

`parse_elemento(s)`

Propósito: Convierte una línea de texto en un elemento del conjunto.

Entrada: Cadena de texto. Puede ser un valor simple o una cadena con notación de conjunto anidado, como {a,b}.

Salida: String primitivo o frozenset (para representar subconjuntos anidados).

Detalle: Detecta si la cadena inicia y termina con llaves {}. Si es así, divide el interior por comas y construye un frozenset. Soporta anidamiento a 2 niveles.

`cargar_conjunto_desde_texto(texto)`

Propósito: Parsea texto multilínea en un conjunto Python.

Entrada: Cadena multilínea donde cada línea es un elemento. Las líneas que comienzan con # son comentarios y se ignoran.

Salida: Objeto set de Python con los elementos parseados.

Detalle: Itera línea por línea, limpia espacios y llama a `parse_elemento()` por cada línea válida. Permite construir conjuntos de forma legible desde archivos de texto.

`formatear_elemento(e)`

Propósito: Convierte un elemento a su representación legible para el usuario.

Entrada: Elemento que puede ser un string o un frozenset.

Salida: Cadena de texto formateada. Los frozensets se muestran como {a, b} con elementos ordenados.

Detalle: Útil para la capa de presentación. Garantiza que los elementos anidados se muestren de forma consistente sin importar el orden interno del frozenset.

`formatear_conjunto(s)`

Propósito: Convierte un conjunto completo a su representación legible.

Entrada: Objeto set de Python.

Salida: Cadena formateada del tipo {elem1, elem2, ...}. Si el conjunto está vacío, retorna el símbolo $\emptyset$.

Detalle: Ordena los elementos y los formatea individualmente usando `formatear_elemento()`. Garantiza una salida determinista para conjuntos del mismo contenido.

`evaluar_expresion_conjuntos(expr_str, conjuntos)`

Propósito: Evalúa una expresión de conjuntos escrita en texto libre de forma segura.

Entrada: Cadena de expresión (ej: $(A \mid B) - C$) y diccionario de conjuntos {nombre: set}.

Salida: Objeto set con el resultado de la expresión.

Detalle: Traduce alias de palabras clave (union, inter, symdiff, diff) a operadores de Python. Maneja el complemento $\sim X$ como $(U - X)$. Usa `eval()` con un namespace controlado y sin builtins para prevenir ejecución de código malicioso.

`operacion_guiada(conj_a, conj_b, operacion)`

Propósito: Ejecuta una operación binaria predefinida entre dos conjuntos.

Entrada: Dos objetos set y el nombre de la operación como string.

Salida: Tupla (resultado: set, símbolo: str) donde el símbolo es el Unicode matemático correspondiente ($\cup$, $\cap$, $-$, $\blacksquare$).

Detalle: Usa un diccionario de operaciones para mapear nombres a expresiones Python. Si la operación no está definida, lanza `ValueError`.

MÓDULO 4: FRONTEND — TAB CONJUNTOS

`_construir_tab_conjuntos()`

Propósito: Construye el layout completo del tab Teoría de Conjuntos.

Entrada: Ninguna. Se ejecuta en `__init__`.

Salida: Paneles, tarjetas y widgets renderizados en el tab.

Detalle: Divide el tab en dos columnas: panel izquierdo de gestión de conjuntos (con `CTkScrollableFrame`) y panel derecho de operaciones y resultados. Inicializa el diccionario interno `_conjuntos` con los cuatro conjuntos base: U, A, B, C.

`_agregar_tarjeta(nombre)`

Propósito: Crea y registra visualmente una tarjeta de edición para un conjunto.

Entrada: Nombre del conjunto como string (ej: 'A', 'D').

Salida: Frame insertado en el scroll con `TextBox`, botones Cargar archivo y Aplicar texto.

Detalle: Genera dinámicamente la tarjeta con diseño oscuro (`#16213E`). Almacena referencia en el diccionario `_tarjetas_conj` para acceso posterior. La tarjeta incluye un label con el nombre y la cardinalidad del conjunto.

`_agregar_conjunto()` / `_eliminar_ultimo_conjunto()`

Propósito: Gestiona la adición y eliminación dinámica de conjuntos (BONO ilimitados).

Entrada: Ninguna.

Salida: Nuevo conjunto disponible en el sistema o eliminación del último conjunto no protegido.

Detalle: `_agregar_conjunto()` busca el siguiente nombre disponible en el alfabeto y crea la tarjeta. Los conjuntos U, A, B y C están protegidos y no pueden eliminarse.

`_construir_modos_guiado()`

Propósito: Construye el sub-tab de operación guiada con `ComboBoxes`.

Entrada: Ninguna.

Salida: Tres ComboBoxes: Conjunto A, Operación, Conjunto B.

Detalle: Usa CtkComboBox con los nombres de conjuntos disponibles. El ComboBox de operación lista las cuatro operaciones principales con notación matemática Unicode.

`_construir_modos_expresion()`

Propósito: Construye el sub-tab de expresión libre con un campo de texto.

Entrada: Ninguna.

Salida: Entry con placeholder y etiqueta de sintaxis de referencia.

Detalle: Permite al usuario escribir expresiones arbitrarias como $(A \mid B) - C \wedge D$. La etiqueta de sintaxis muestra los operadores aceptados: $\mid$, $\&$, $-$, $\wedge$, $\sim$ para complemento.

`_calcular_conjuntos()`

Propósito: Orquesta el proceso de cálculo según el modo activo.

Entrada: Ninguna. Lee el modo del CtkTabview activo.

Salida: Resultado renderizado en el panel derecho.

Detalle: Detecta si el modo activo es Guiado o Expresión Libre. Invoca el backend correspondiente y llama a `_mostrar_resultado_conjuntos()` o captura errores con `_mostrar_error_conj()`.

`_mostrar_resultado_conjuntos(expresion, resultado)`

Propósito: Renderiza el resultado de una operación con tabla de pertenencia.

Entrada: Expresión usada como string y conjunto resultado como set.

Salida: Widgets en el frame de resultados: encabezado, conjunto formateado, cardinalidad y tabla de pertenencia.

Detalle: Construye la tabla de pertenencia usando la unión del conjunto resultado con el universo U. Coloriza ✓ Sí en verde (#2CC985) y ✗ NO en rojo (#FF5555).

ALGORITMO DE EVALUACIÓN DE EXPRESIONES

El algoritmo se implementa en `evaluar_expresion_conjuntos()`. Su objetivo es interpretar y ejecutar operaciones de teoría de conjuntos expresadas como texto libre, de forma segura y extensible.

1. Normalización de alias

La expresión recibida es preprocesada mediante expresiones regulares. Las palabras clave `union`, `inter`, `symdiff` y `diff` son reemplazadas por sus operadores Python equivalentes (`|`, `&`, `^`, `-`). El operador de complemento `~X` es transformado a `(U - X)` usando un lambda de regex.

2. Construcción del namespace

Se construye un diccionario Python que mapea cada nombre de conjunto (`{A, B, C, U, ...}`) a su objeto set correspondiente. Este namespace actúa como entorno de ejecución controlado.

3. Evaluación segura con `eval()`

Se invoca `eval()` pasando el namespace controlado como `locals` y un diccionario vacío como `globals` (`{'__builtins__': {}}`). Esto impide el acceso a funciones del sistema, módulos o builtins de Python, mitigando el riesgo de inyección de código.

4. Validación del resultado

Se verifica que el resultado de la evaluación sea instancia de set o frozenset. Si no lo es (por ejemplo si el usuario escribió una expresión aritmética), se lanza un `ValueError` descriptivo.

5. Propagación de errores

Los errores de `KeyError` (conjunto no definido), `SyntaxError` (expresión mal escrita) y excepciones genéricas son capturados y relanzados como `ValueError` con mensajes claros, que el frontend muestra al usuario.

ALGORITMO DE OPERACIÓN GUIADA

El algoritmo se implementa en `operacion_guiada()`. Su objetivo es ejecutar operaciones binarias de forma predefinida, eliminando la necesidad de que el usuario conozca la sintaxis de expresiones.

Operación	Símbolo	Operador Python	Descripción
Unión	$\cup$	<code>a b</code>	Todos los elementos de A y B sin repetición
Intersección	$\cap$	<code>a & b</code>	Solo elementos presentes en ambos conjuntos
Diferencia	$-$	<code>a - b</code>	Elementos de A que no están en B
Diferencia Simétrica	$\oplus$	<code>a ^ b</code>	Elementos en A o B pero no en ambos

El complemento de un conjunto ($\sim X$) no está disponible en el modo Guiado porque requiere un universo de referencia U y opera sobre un único operando. Esta operación está disponible exclusivamente en el modo Expresión Libre usando la sintaxis $\sim X$, que el parser traduce a $(U - X)$.

MANEJO DE ERRORES (PARTE II)

El módulo de conjuntos extiende la política de Zero Trust establecida en la Parte I. Cada capa del sistema valida su entrada antes de procesarla.

Tipo de error	Origen	Manejo
Conjunto no definido	<code>evaluar_expresion_conjuntos()</code>	KeyError → ValueError con mensaje descriptivo
Sintaxis inválida	<code>eval() interno</code>	SyntaxError capturado, se muestra ■ en la UI
Resultado no es set	<code>evaluar_expresion_conjuntos()</code>	ValueError: La expresión no produjo un conjunto válido
Archivo ilegible	<code>_cargar_archivo()</code>	Exception genérica → messagebox.showerror
Parseo de elemento	<code>cargar_conjunto_desde_texto()</code>	Silenciado por tipo (frozenset vs str), nunca falla
Operación desconocida	<code>operacion_guiada()</code>	ValueError con nombre de la operación no reconocida

ESTRUCTURAS DE DATOS UTILIZADAS

set (conjunto Python)

Estructura principal del módulo. Cada conjunto definido por el usuario se almacena como un objeto set. Los sets de Python ofrecen de forma nativa las operaciones $|$, $&$, $-$ y $^$ que se corresponden directamente con las operaciones de teoría de conjuntos.

frozenset (subconjunto anidado)

Usado para representar elementos que a su vez son conjuntos (anidamiento a 2 niveles). Los frozensets son hashables, lo que permite que sean elementos de un set padre. Se generan en `parse_elemento()` cuando se detecta la notación $\{a,b\}$.

dict (_conjuntos)

Diccionario interno de la clase SuiteLogica que mapea nombre de conjunto (string) a objeto set. Es la única fuente de verdad del estado de los conjuntos en la aplicación. Se actualiza en tiempo real al aplicar texto o cargar archivos.

dict (_tarjetas_conj)

Diccionario que mapea nombre de conjunto a sus widgets visuales (frame, textbox, label). Permite acceder y actualizar la UI de cada tarjeta individualmente sin recorrer toda la jerarquía de widgets.

tuple (resultado de operacion_guiada)

La función `operacion_guiada()` devuelve una tupla (set, str) que empaqueta el resultado matemático con el símbolo Unicode para su uso inmediato en la interfaz.

CONCLUSIÓN TÉCNICA

La Parte II del proyecto integrador LyED Toolkit demuestra cómo una arquitectura modular bien definida permite extender un sistema existente sin romper la funcionalidad previa. La incorporación del módulo de Teoría de Conjuntos reutilizó la infraestructura visual y el patrón de diseño establecido en la Parte I, reduciendo el costo de integración.

El backend de conjuntos destaca por su diseño funcional: todas sus funciones son puras, sin estado compartido, lo que facilita su prueba y mantenimiento. La seguridad del evaluador de expresiones libres fue garantizada mediante un namespace controlado que elimina el acceso a builtins de Python.

La implementación del bono de conjuntos ilimitados (gestión dinámica de tarjetas) y la tabla de pertenencia por elemento demuestran un diseño orientado a la usabilidad que va más allá de los requisitos mínimos. En conjunto, el sistema integra con éxito lógica proposicional y teoría de conjuntos en una sola herramienta educativa cohesiva.